

Parcel Scanner Sim – Testy wydajnościowe

Spis treści

1. Cel testów	3
2. Środowisko	3
3. Badane metryki w testach.....	3
4. Scenariusz testów	4
5. Interpretacja wyników oraz defekty	8

1. Cel testów

Celem testów jest zweryfikowanie przypadków testowych pod kątem wpływu na wydajność aplikacji, pomimo, iż zamysł aplikacji był dla małych sortowni wysyłkowych, chcemy sprawdzić czy ilość elementów oraz ruch sieciowy będzie miał wpływ na wydajność aplikacji w aspektach jak wykorzystanie zasobów sprzętowych czy czas odpowiedzi.

2. Środowisko

Testy wydajnościowe zostaną przeprowadzone w środowisku testowym na fizycznym urządzeniu podłączonym do Android Studio wraz z narzędziem Profiler w celu zbadania zużycia zasobów sprzętowych w różnych warunkach. W testach wydajnościowych często stosowana jest praktyka, że jeżeli aplikacja działa wydajnie na starszym urządzeniu, to poradzi sobie na nowszych.

Ważnym krokiem w celu zdobycia rzetelnych wyników jest wyłączenie globalne animacji przez ADB:

- `adb shell settings put global window_animation_scale 0`
- `adb shell settings put global transition_animation_scale 0`
- `adb shell settings put global animator_duration_scale 0`

3. Badane metryki w testach

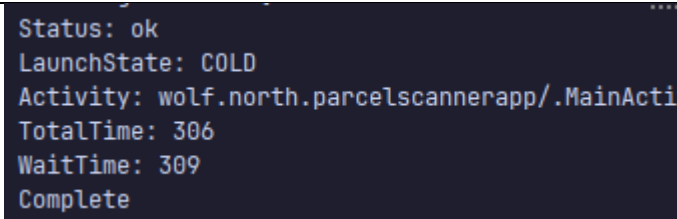
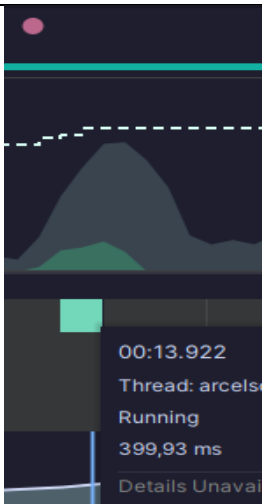
Biorąc pod uwagę rozmiar aplikacji i ilość funkcjonalności nie zostały wykorzystane specjalistyczne narzędzia do automatyzacji testów wydajnościowych jak UI Automator czy macrobenchmark. Wykorzystam wbudowane w środowisko narzędzie Android Profiler w oparciu o poprawne metryki wyznaczone przez Google oraz zbadanie najważniejszych scenariuszy testowych.

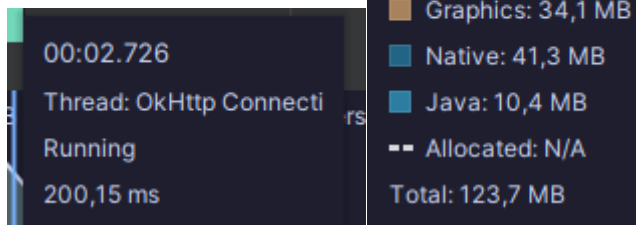
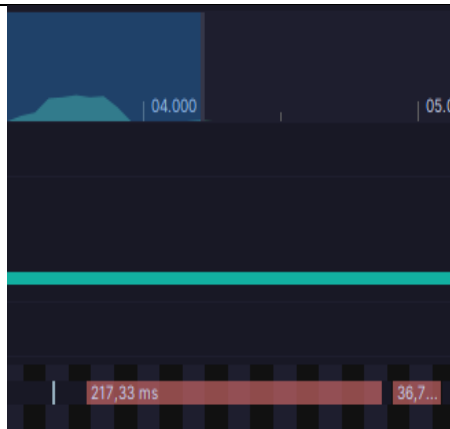
- Cold, Warm, Hot Start
- 16.6ms rule – czyli metryka płynności renderowania (Jank & Frame Rate)
- Metryki pamięci i zarządzania pamięcią
- Obciążenie CPU

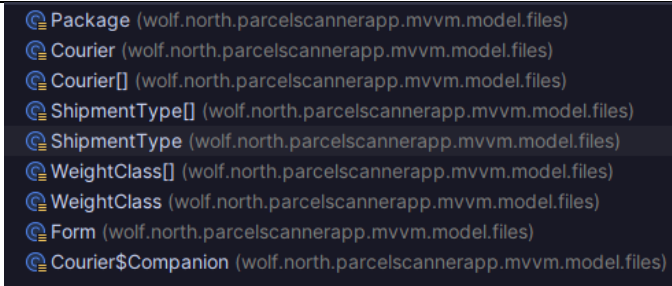
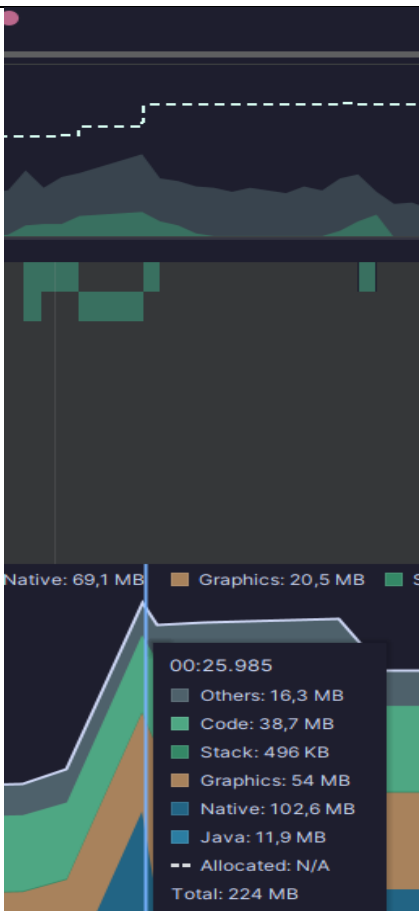
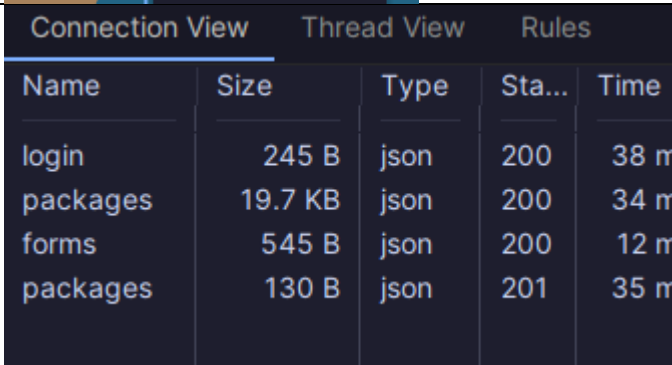
4. Scenariusz testów

ID Test Case	Nazwa	Opis kroków	Badane Metryki	Oczekiwane rezultaty
TC-PERF-01	Uruchomienie aplikacji (Cold Start)	1. Zabicie procesu w ADB 2. Kliknięcie lub uruchomienie aplikacji w ADB 3. Oczekiwanie na ekran logowania	Czas startu	Według Android Vitality czas powinien być mniejszy niż 5000ms oraz brak białych/zawieszonych ekranów
TC-PERF-02	Poprawne zalogowanie użytkownika z przejściem do Home.	1. Uruchomienie aplikacji 2. Zalogowanie do systemu	Czas odpowiedzi API oraz obciążenie CPU	Czas od kliknięcia zaloguj do załadowania ekranu Home < 2000ms. Brak zatorów i wąskich gardeł w głównym wątku aplikacji przy przetwarzaniu danych.
TC-PERF-03	Poprawne zalogowanie użytkownika z przejściem do Home z przypisaną dużą listą	1. Uruchomienie aplikacji 2. Zalogowanie do systemu	Czas odpowiedzi API (Network) + Obciążenie CPU (deserializacja JSON).	Czas od kliknięcia "Zaloguj" do pojawienia się danych na Home < 2000ms (zależnie od sieci). Brak zatorów w wątku głównym (Main Thread) podczas przetwarzania danych.
TC-PERF-04	Przejście zalogowanego użytkownika do ekranu historii.	1. Uruchom aplikacji 2. Zaloguj użytkownika 3. Przejdź do ekranu historii	Czas renderowania UI (UI Rendering).	Płynne przejście, brak szarpnięć. Renderowanie listy < 16ms na klatkę (dla pustej listy) lub < 100ms (dla dużej listy - pierwsze renderowanie).
TC-PERF-05	Przejście zalogowanego użytkownika z przypisaną dużą listą do ekranu historii	1. Uruchom aplikacji 2. Zaloguj użytkownika 3. Przejdź do ekranu historii	Czas renderowania UI, liczba Janky Frames.	Płynne przejście, brak szarpnięć. Ekran nie powinien zaczynać się pomimo ładowanie dużej listy.
TC-PERF-06	Badanie płynności listy przy dużym obciążeniu (Heavy list testing)	1. Zaloguj się jako użytkownik z przypisaną dużą listą 2. Wejdź w ekran historii 3. Szybkie przewijanie listy w górę i dół przez 10 sekund.	Rendering (Frame Rates)	Stosując się do zasady 16.6ms oraz wyznaczając średnią ilość FPS jako 60. Liczba klatek opóźnionych typu Jank < 5%. Brak zamrożonych klatek (Frozen frames z czasem > 700ms).
TC-PERF-07	Badanie zużycia pamięci przy ładowaniu danych w ekranie historii	1. Zaloguj się jako użytkownik z przypisaną dużą listą 2. Wejdź w ekran historii	Pamięć a dokładnie Memory Allocation w Profiler	Brak ciągłych wzrostów, brak memory leaków. Wymuszone działanie Garbage Collector nie zmienia wartości w pamięci przy

		3. Wróć do ekranu Home 4. Wymuś działanie Garbage Collector w Profiler.		pobieraniu/wyświetlaniu listy
TC-PERF-08	Badanie zużycia zasobów przez Skaner (Package Scanner)	1. Zaloguj się jako użytkownik 2. Wejdź w tryb skanowania paczek 3. Zeskanuj kod kreskowy poprawnej paczki 4. Kliknij „zapisz paczkę” w bottom sheet z wynikiem.	Obciążenie CPU oraz Pamięć	Aplikacja reaguje płynnie, czas od wykrycia kodu do wyświetlania wyniku < 1000ms. Brak przegrzania urządzenia wyznaczone przez Thermal Status z wartością OK.
TC-PERF-09	Badanie zasobów w przypadku poprawnego zeskanowania i zapisania formularza w aplikacji.	1. Zaloguj się jako użytkownik 2. Włącz skaner paczek i zeskanuj poprawnie paczkę 3. Kliknij „zapisz” w bottom sheet z wyświetlonymi danymi z skanowania.	Network Usage	Zapytanie POST z formularzem nie blokuje głównego UI, UI zostaje responsywne podczas wysyłania danych.

Nazwa przypadku	Opis	Wynik	Interpretacja
TC-PERF-01	Badanie czasu uruchomienia aplikacji w trybie Cold Start		Ponowienie prób uruchomienia w trybie Cold wyświetliło wyniki w przedziale 250-380ms.
TC-PERF-02	Poprawne zalogowanie użytkownika z przejściem do Home.		Poprawne zalogowanie użytkownika i przejście na ekran Home zostało wykonane w przedziałach od 389-410ms.

TC-PERF-03	Poprawne zalogowanie użytkownika z przejściem do Home z przypisaną dużą listą		Poprawne zalogowanie użytkowa i przejście na ekran Home z dużą listą w API zostało wykonane w akceptowalnych przedziałach czasowych a obciążenie nie zwiększyło się drastycznie.																																																																																										
TC-PERF-04	Przejście zalogowane go użytkownika do ekranu historii.		Przejście zalogowane użytkownika do ekranu historii zostało wykonane w akceptowalnych przedziałach od 298-371ms																																																																																										
TC-PERF-05	Przejście zalogowane go użytkownika z dużą listą do ekranu historii	Time Range00:01.789 - 00:02.291 Duration502,32 ms Data TypeThread ID21533 States <table><thead><tr><th>Thread State</th><th>Duration</th><th>%</th><th>Occurrences</th></tr></thead><tbody><tr><td>Running</td><td>301,78 ms</td><td>60,08%</td><td>48</td></tr><tr><td>Sleeping</td><td>96,22 ms</td><td>19,16%</td><td>11</td></tr><tr><td>No thread activity</td><td>42,95 ms</td><td>8,55%</td><td>1</td></tr><tr><td>Runnable</td><td>3,05 ms</td><td>0,61%</td><td>19</td></tr><tr><td>Waiting</td><td>1,44 ms</td><td>0,29%</td><td>18</td></tr></tbody></table> Longest running events (top 10) <table><thead><tr><th>Start Time</th><th>Name</th><th>Wall Dura...</th><th>Wall Self ...</th><th>CPU Dur...</th><th>CPU Self ...</th></tr></thead><tbody><tr><td>00:01.973</td><td>Choreogr...</td><td>211,71 ms</td><td>597 μs</td><td>209,47 ms</td><td>81 μs</td></tr><tr><td>00:02.009</td><td>traversal</td><td>175,98 ms</td><td>1,02 ms</td><td>175,3 ms</td><td>342 μs</td></tr><tr><td>00:02.009</td><td>measure</td><td>125,99 ms</td><td>812 μs</td><td>125,99 ms</td><td>812 μs</td></tr><tr><td>00:02.010</td><td>Android...</td><td>124,95 ms</td><td>25,92 ms</td><td>124,95 ms</td><td>25,92 ms</td></tr><tr><td>00:02.192</td><td>Choreogr...</td><td>38,32 ms</td><td>56 μs</td><td>37,74 ms</td><td>80 μs</td></tr><tr><td>00:01.973</td><td>animation</td><td>35,07 ms</td><td>79 μs</td><td>34,03 ms</td><td>370 μs</td></tr><tr><td>00:01.973</td><td>Recomp...</td><td>34,99 ms</td><td>4,36 ms</td><td>33,66 ms</td><td>3,03 ms</td></tr><tr><td>00:02.155</td><td>draw</td><td>30,12 ms</td><td>139 μs</td><td>30,12 ms</td><td>139 μs</td></tr><tr><td>00:02.155</td><td>Record V...</td><td>29,98 ms</td><td>29,88 ms</td><td>29,98 ms</td><td>29,88 ms</td></tr><tr><td>00:02.192</td><td>animation</td><td>26,7 ms</td><td>154 μs</td><td>26,7 ms</td><td>154 μs</td></tr></tbody></table>	Thread State	Duration	%	Occurrences	Running	301,78 ms	60,08%	48	Sleeping	96,22 ms	19,16%	11	No thread activity	42,95 ms	8,55%	1	Runnable	3,05 ms	0,61%	19	Waiting	1,44 ms	0,29%	18	Start Time	Name	Wall Dura...	Wall Self ...	CPU Dur...	CPU Self ...	00:01.973	Choreogr...	211,71 ms	597 μs	209,47 ms	81 μs	00:02.009	traversal	175,98 ms	1,02 ms	175,3 ms	342 μs	00:02.009	measure	125,99 ms	812 μs	125,99 ms	812 μs	00:02.010	Android...	124,95 ms	25,92 ms	124,95 ms	25,92 ms	00:02.192	Choreogr...	38,32 ms	56 μs	37,74 ms	80 μs	00:01.973	animation	35,07 ms	79 μs	34,03 ms	370 μs	00:01.973	Recomp...	34,99 ms	4,36 ms	33,66 ms	3,03 ms	00:02.155	draw	30,12 ms	139 μs	30,12 ms	139 μs	00:02.155	Record V...	29,98 ms	29,88 ms	29,98 ms	29,88 ms	00:02.192	animation	26,7 ms	154 μs	26,7 ms	154 μs	Przejście zalogowane użytkownika z dużą listą paczek do ekranu historii zostało wykonane w akceptowalnych przedziałach od 298-371ms
Thread State	Duration	%	Occurrences																																																																																										
Running	301,78 ms	60,08%	48																																																																																										
Sleeping	96,22 ms	19,16%	11																																																																																										
No thread activity	42,95 ms	8,55%	1																																																																																										
Runnable	3,05 ms	0,61%	19																																																																																										
Waiting	1,44 ms	0,29%	18																																																																																										
Start Time	Name	Wall Dura...	Wall Self ...	CPU Dur...	CPU Self ...																																																																																								
00:01.973	Choreogr...	211,71 ms	597 μs	209,47 ms	81 μs																																																																																								
00:02.009	traversal	175,98 ms	1,02 ms	175,3 ms	342 μs																																																																																								
00:02.009	measure	125,99 ms	812 μs	125,99 ms	812 μs																																																																																								
00:02.010	Android...	124,95 ms	25,92 ms	124,95 ms	25,92 ms																																																																																								
00:02.192	Choreogr...	38,32 ms	56 μs	37,74 ms	80 μs																																																																																								
00:01.973	animation	35,07 ms	79 μs	34,03 ms	370 μs																																																																																								
00:01.973	Recomp...	34,99 ms	4,36 ms	33,66 ms	3,03 ms																																																																																								
00:02.155	draw	30,12 ms	139 μs	30,12 ms	139 μs																																																																																								
00:02.155	Record V...	29,98 ms	29,88 ms	29,98 ms	29,88 ms																																																																																								
00:02.192	animation	26,7 ms	154 μs	26,7 ms	154 μs																																																																																								
TC-PERF-06	Badanie płynności listy przy dużym obciążeniu (Heavy list testing)	** Graphics info for pid 16411 [wolf.north.parcelscannerapp] Stats since: 693968988120ns Total frames rendered: 506 Janky frames: 98 (19.37%)	Przewijanie całej listy z 176 elementami zapewniło wyniki Janky Frames w przedziale 18-19.5%																																																																																										

TC-PERF-07	Badanie zużycia pamięci przy ładowaniu danych w ekranie historii		Elementy historii i ich obiekty w pamięci zostają zebrane przez garbage collector po przejściu na inny ekran, brak wycieków pamięci.
TC-PERF-08	Badanie zużycia zasobów przez Skaner (Package Scanner)		Proces poprawnego zeskanowania oraz zapisania paczki nie zwiększył drastycznie zużycia pamięci oraz procesora, zwiększyła się liczba wątków ze względu na uruchomienie procesów CameraX oraz wątków do parsowania wyników oraz Retrofit do poprawnego wysłania żądania do API.
TC-PERF-09	Badanie zasobów w przypadku poprawnego zeskanowania i zapisania paczki w aplikacji.		Poprawne zeskanowanie i zapisanie paczki w aplikacji wysłało żądanie POST, czasy odpowiedzi jak i inicjowania danych z API oraz zapisywania są w

			poprawnych ramach czasowych.
--	--	--	------------------------------------

5. Interpretacja wyników oraz defekty

Proces testowania wydajnościowego przeszedł pomyślnie, nie zostały znalezione defekty które wymagałyby natychmiastowej naprawy, jedynie w przypadku testowym TC-PERF-06 można zauważyć, że liczba Janky Frames była w przedziałach 18-19.5% co należałoby zgłosić z niskim priorytetem, lecz dla MVP jest to akceptowalne.